

Mobile Learning Application Development



Dart Programming Language Lecture 2

Aditya Rajbongshi
Assistant Professor

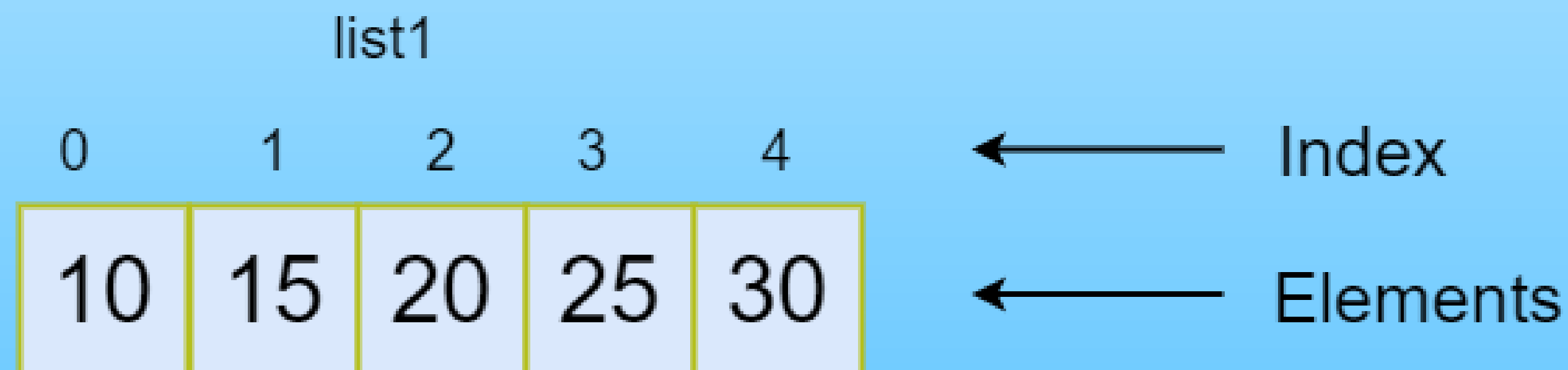
Dept. of Educational Technology and Engineering
University of Frontier Technology, Bangladesh



Dart Data Type: List

- Dart List is similar to an **array**, which is the ordered collection of the **objects**.
- The Dart list looks like the JavaScript array literals. The syntax of declaring the list is given below.

```
var list1 = [10, 15, 20, 25, 25]
```





Dart Data Type: List

- The Dart list can be categorized into two types –

✓ **Fixed Length List:** The fixed-length lists are defined with the specified length. We cannot change the size at runtime. The syntax is given below.

- Syntax - Create the list of fixed-size
- **`var list_name = List<type>.filled(size, 0)`**

```
void main() {  
    var list1=List<int>.filled(5,0);  
    list1[0] = 10;  
    list1[1] = 11;  
    list1[2] = 12;  
    list1[3] = 13;  
    list1[4] = 14;  
    print(list1);  
}
```




Dart Data Type: List

- The Dart list can be categorized into two types –
 - ✓ **Growable List:** The fixed-length lists are defined with the specified length. We cannot change the size at runtime. The syntax is given below.
 - Syntax -
 - `var list_name = [val1, val2, val3]`
 - `var list_name = new List()`

```
void main() {  
    var list1 = [10,11,12,13,14,15];  
    print(list1);  
}
```



Dart Data Type: List

- **List Properties**

- **first** - returns the first element case
- **isEmpty** - returns true if the list is empty
- **isNotEmpty** - returns true if the list has at least one element
- **length** - returns the length of the list
- **last** - returns the last element of the list
- **reversed** - returns a list in reverse order
- **Single** - it checks if the list has only one element and returns it



Dart Data Type: List

- Dart provides **four methods** which are used to insert the elements into the lists. These methods are given below.
 - **add()**
 - **addAll()**
 - **insert()**
 - **insertAll()**

```
1. void main() {  
    var odd_list = [1,3,5,7,9];  
    print(odd_list);  
    odd_list.add(11);  
    print(odd_list);  
}
```

```
2. void main() {  
    var odd_list = [1,3,5,7,9]  
    print(odd_list);  
    odd_list.addAll([11,13,14]);  
    print(odd_list);  
}
```




Dart Data Type: List

- Dart provides four methods which are used to insert the elements into the lists. These methods are given below.

- add()
- addAll()
- insert()
- insertAll()

```
3. void main(){  
    List lst = [3,4,2,5];  
    print(lst);  
    lst.insert(2,10);  
    print(lst);  
}
```

```
4. void main(){  
    List lst = [3,4,2,5];  
    print(lst);  
    lst.insertAll(0,[6,7,10,9]);  
    print(lst);  
}
```



Dart Data Type: List

- The Dart provides following functions

to remove the list elements.

- `remove()`
- `removeAt()`
- `removeLast()`
- `removeRange()`

```
void main(){  
    var list1 = [10,15,20,25,30];  
    print("List before remove element :  
    ${list1}");  
    list1.remove(20) ;  
    print("List after removing element :  
    ${list1}");  
}
```




Dart Data Type: Sets

- The Dart Set is the unordered collection of the different values of the **same type**.
- It has much functionality, which is the same as an array, but it is **unordered**.
- Set **doesn't** allow storing the **duplicate** values.
- The set must contain unique values.
- The set can be declared by using the **{}** **curly braces** proceeded by a type argument, or declare the variable type Set with curly braces **{}**



Dart Data Type: Sets

- The syntax of declaring set is given below.

✓ `var setName = <type>{};`

Or

✓ `Set<type> setname = {};`

```
void main(){
```

```
    print("Initializing the Set");
```

```
    var names = <String>{"James","  
    Ricky", "Devansh","Adam"};
```

```
    print(names);
```

```
}
```



Dart Data Type: Set

- **Set Properties:**

- **first** - returns the first element case
- **isEmpty** - returns true if the list is empty
- **isNotEmpty** - returns true if the list has at least one element
- **length** - returns the length of the set
- **last** - returns the last element of the set
- **hashCode** - returns the hashCode for the corresponding object
- **Single** - it checks if the list has only one element and returns it



Dart Data Type: Set

- **Set Methods:**

- `add()`
- `addAll()`
- `elementAt()`
- `remove()`
- `clear()`

```
void main(){  
    print("Insert element into the Set");  
    var names = {"James","Ricky","Devansh","Adam"};  
  
    // Declaring empty set  
    var emp = <String>{};  
    emp.add("Jonathan");  
    print(emp);  
  
    // Adding multiple elements  
    emp.addAll(names);  
    print(emp);  
}
```



Dart Data Type: Set

- **Set Methods:**

- add()
- addAll()
- **elementAt()**
- remove()
- clear()

```
void main(){  
    print("Access element from the Set");  
    var names = {"James","Ricky","Devansh","Adam"};  
    print(names);  
  
    var x = names.elementAt(3);  
    print(x);  
}
```

Dart Data Type: Sets

```
void main() {  
  
    var x = {"A","B","C"};  
    x.add("D");  
    print(x);  
  
    x.addAll({"E","F"});  
    print(x);  
    var elem=x.elementAt(3);  
    print(elem);  
  
    x.remove("E");  
    print(x);  
  
    x.clear();  
    print(x);  
}
```




Dart Data Type: Map

- The maps type is used to store values in **key-value pairs**. Each key is associated with its value.
- The key and value can be **any type**. In Map, the key must be **unique**, but a value can occur multiple times.
- The Map is defined by using **curly braces ({})**, and comma separates each pair.
- Dart Map is an object that stores data in the form of a key-value pair.
- Each value is associated with its key, and it is used to access its **corresponding value**.
- Both keys and values can be any type.
- In Dart Map, each key must be unique, but the same value can occur multiple times.



Dart Data Type: Map

- Dart Map can be defined in two methods.
 - **Using Map Literal**
 - To declare a Map using map literal, the key-value pairs are enclosed within the curly braces "{}" and separated by the commas.
 - **Using Map Constructor**
 - To declare the Dart Map using map constructor can be done in two ways.
 - First, declare a map using map() constructor. Second, initialize the map.



Dart Data Type: Map

- Dart Map can be defined in two methods.
 - Using Map Literal
 - To declare a Map using map literal, the **key-value pairs are enclosed** within the curly braces "{}" and separated by the commas.
 - Using Map Constructor
 - To declare the Dart Map using map constructor can be done in two ways.
 - First, declare a map using map() constructor. Second, initialize the map.

```
var map_name = {key1:value1, key2:value2  
[.....,key_n: value_n]}
```

```
void main() {  
    var student = {'name':'Tom','age':'23'};  
    print(student);  
}
```



Dart Data Type: Map

- Dart Map can be defined in two methods.
 - Using Map Literal
 - To declare a Map using map literal, the key-value pairs are enclosed within the curly braces "{}" and separated by the commas.
 - Using Map Constructor
 - To declare the Dart Map using map constructor can be done in **two ways**.
 - First, declare a map using map() constructor. Second, initialize the map.

```
var map_name = new map()
```

```
void main() {  
    var student = new Map();  
    student['name'] = 'Tom';  
    student['age'] = 23;  
    student['course'] = 'B.tech';  
    student['Branch'] = 'Computer Science';  
  
    print(student);  
}
```



Dart Data Type: Map

- **Properties**

- **keys** - Returns an iterable object representing keys
- **values** - Returns an iterable object representing values
- **length** - Returns the size of the Map
- **isEmpty** - Returns true if the Map is an empty Map
- **isNotEmpty** - Returns true if the Map is an empty Map



Dart Data Type: Map

● Methods

- **addAll()** - It adds multiple key-value pairs of other. The syntax is given below.
- **clear()** - It eliminates all pairs from the map.
- **remove()** - It removes the key and its associated value if it exists in the given map.
- **forEach()** - Applies f to each key-value pair of the map.

```
void main() {  
    Map student = {'name':'Tom','age': 23};  
  
    print('Map :${student}');  
  
    student.addAll({'dept':'Civil','email':'tom@xyz.com'});  
    print('Map after adding key-values :${student}');  
}
```



Dart Data Type: Map

- **Methods**

- **addAll()** - It adds multiple key-value pairs of other. The syntax is given below.
- **clear()** - It eliminates all pairs from the map.
- **remove()** - It removes the key and its associated value if it exists in the given map.
- **forEach()** - Applies f to each key-value pair of the map.

```
void main() {  
    Map student = {'name':'Tom','age': 2  
3};  
    print('Map :${student}');  
  
    student.clear();  
    print('Map after removing all key-  
values :${student}');  
}
```



Dart Data Type: Map

- **Methods**

- **addAll()** - It adds multiple key-value pairs of other. The syntax is given below.
- **clear()** - It eliminates all pairs from the map.
- **remove()** - It removes the key and its associated value if it exists in the given map.
- **forEach()** - Applies f to each key-value pair of the map.

```
void main() {  
    Map student = {'name':'Tom','age'  
: 23};  
    print('Map :${student}');  
  
    student.remove('age');  
    print('Map after removing given k  
ey :${student}');  
}
```




Dart Data Type: Map

- **Methods**

- **addAll()** - It adds multiple key-value pairs of other. The syntax is given below.
- **clear()** - It eliminates all pairs from the map.
- **remove()** - It removes the key and its associated value if it exists in the given map.
- **forEach()** - Applies f to each key-value pair of the map.

```
void main() {  
    Map student = {'name':'Tom','age'  
: 23};  
    print('Map :${student}');  
    student.forEach((k,v) => print('${  
k}: ${v}'));  
}
```



Dart Data Type: Runes, Symbol



Home work



Thank you